



INSTITUTO POLITÉCNICO NACIONAL ESCUELA SUPERIOR DE CÓMPUTO

PRÁCTICA 6

CONTADORES

DISEÑO DE SISTEMAS DIGITALES

4CM4

INTEGRANTES:

MONTEALEGRE ROSALES DAVID URIEL
SALAUZ HERNANDEZ NERICK FRANCISCO
VÁZQUEZ BLANCAS CESAR SAID

PROFESOR:

AGUILAR SANCHEZ FERNANDO

INSTITUTO POLITÉCNICO NACIONAL



FECHA DE ENTREGA: 24 DE NOVIEMBRE DE 2024

1. OBJETIVO

El alumno verificará el funcionamiento de los Contadores.

2. MATERIAL Y EQUIPO EMPLEADO

1 GAL 22V10

- ☐ 8 resistencias $330\ \Omega$ a $1/4\ W$
- ☐ 8 resistencias $1\ K\Omega$ a $1/4\ W$
- ☐ 8 Leds del color que guste
- ☐ Dip switch de 8
- ☐ Circuito Oscilador con el CI 555 o 74HC14

3. INTRODUCCIÓN TEÓRICA:

Los **contadores secuenciales** son circuitos digitales que generan secuencias ordenadas de números en una dirección específica (ascendente o descendente) y encuentran aplicación en una amplia gama de sistemas, desde relojes digitales hasta controladores industriales. Su diseño se basa en el uso de **flip-flops**, elementos básicos de almacenamiento, que cambian de estado sincronizados por un reloj.

Clasificación de los Contadores

1. **Contadores** **Ascendentes**
Los contadores ascendentes incrementan su valor de salida en cada pulso de reloj, avanzando de forma ordenada desde un valor mínimo (generalmente 0) hasta un valor máximo definido por el número de bits que lo componen. Por ejemplo, un contador de 4 bits genera una secuencia de 0 a 15 en binario.
2. **Contadores** **Descendentes**
Los contadores descendentes realizan la operación inversa, decreciendo su valor en cada pulso de reloj desde un valor máximo hasta un mínimo. Este tipo de contador es útil en aplicaciones donde se necesita medir tiempos o realizar regresiones.
3. **Contadores** **Ascendentes** **y** **Descendentes**
Estos contadores son configurables, permitiendo alternar entre el modo ascendente y descendente según las necesidades del sistema.
4. **Contadores con Reset y Stop**
 - **Reset:** Este es un mecanismo que permite reiniciar el contador a un valor predeterminado, generalmente 0. Es útil para sincronizar operaciones o reiniciar procesos.

- **Stop:** Permite pausar el conteo en un estado específico. Esta funcionalidad se utiliza para conservar el estado actual sin que el reloj lo altere, útil en aplicaciones donde se requiere mantener un valor hasta un evento externo.

Componentes Principales de los Contadores

1. **Flip-Flops:** Usualmente se utilizan flip-flops tipo D o JK para almacenar el estado del contador.
2. **Circuitos de Reloj:** Sincronizan los cambios de estado en el contador.
3. **Lógica de Control:** Gestiona las funciones de ascenso, descenso, reset y stop.

Funcionamiento General

- **Ascendente:** Cada pulso de reloj genera un incremento en el valor almacenado.
- **Descendente:** Cada pulso genera un decremento en el valor almacenado.
- **Reset:** La entrada de reset borra el contenido y reinicia el contador a un valor inicial.
- **Stop:** Al activarse, el contador deja de responder a los pulsos del reloj.

Aplicaciones

1. **Contadores de Tiempo:** En cronómetros o relojes digitales.
2. **Control de Procesos:** Para realizar un seguimiento del número de ciclos completados.
3. **Medición de Frecuencia:** Contadores usados como divisores de frecuencia.
4. **Automatización Industrial:** Control de procesos y seguimiento de tareas.

El diseño y análisis de contadores requieren un entendimiento profundo de la lógica digital, permitiendo configurar comportamientos específicos a través de combinaciones de puertas lógicas y flip-flops. En esta práctica se abordarán ejemplos prácticos de implementación y uso de contadores ascendentes y descendentes con funcionalidades de reset y stop.

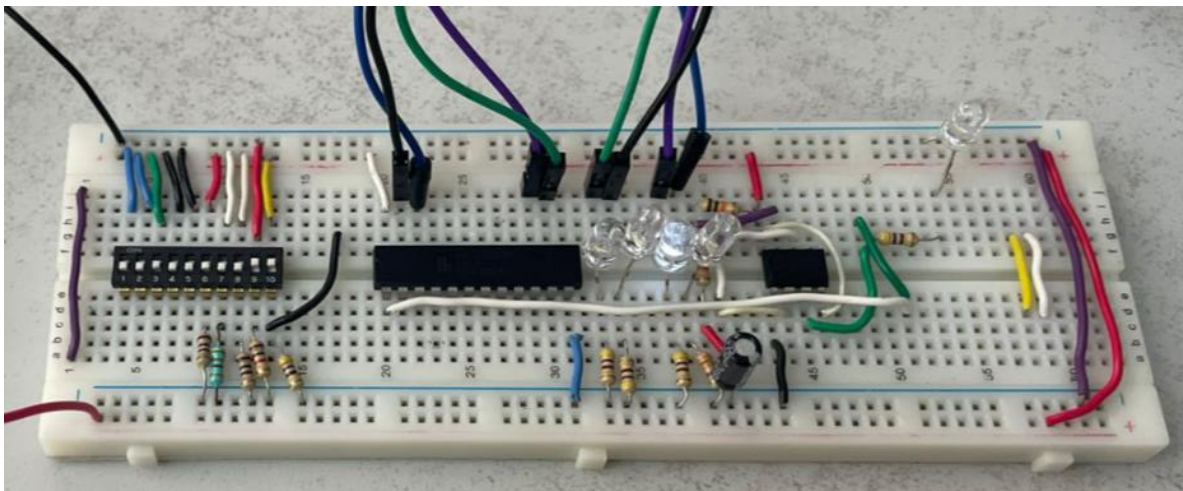
DESARROLLO

contador ascendente de 4 bits.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
entity cont4 is
    port (clk: in std_logic;
          Q: inout std_logic_vector(3 downto 0));
end cont4;
architecture arqcont of cont4 is
begin
    process (clk)
    begin
        if (clk'event and clk = '0') then
            Q <= Q + 1;
        end if;
    end process;
end arqcont;
```

C22V10

clk	=	1	24 *	not used
not used	*	2	23	= q(2)
not used	*	3	22	= q(0)
not used	*	4	21 *	not used
not used	*	5	20 *	not used
not used	*	6	19 *	not used
not used	*	7	18 *	not used
not used	*	8	17 *	not used
not used	*	9	16 *	not used
not used	*	10	15	= q(1)
not used	*	11	14	= q(3)
not used	*	12	13 *	not used



contador ascendente/descendente de 4 bits

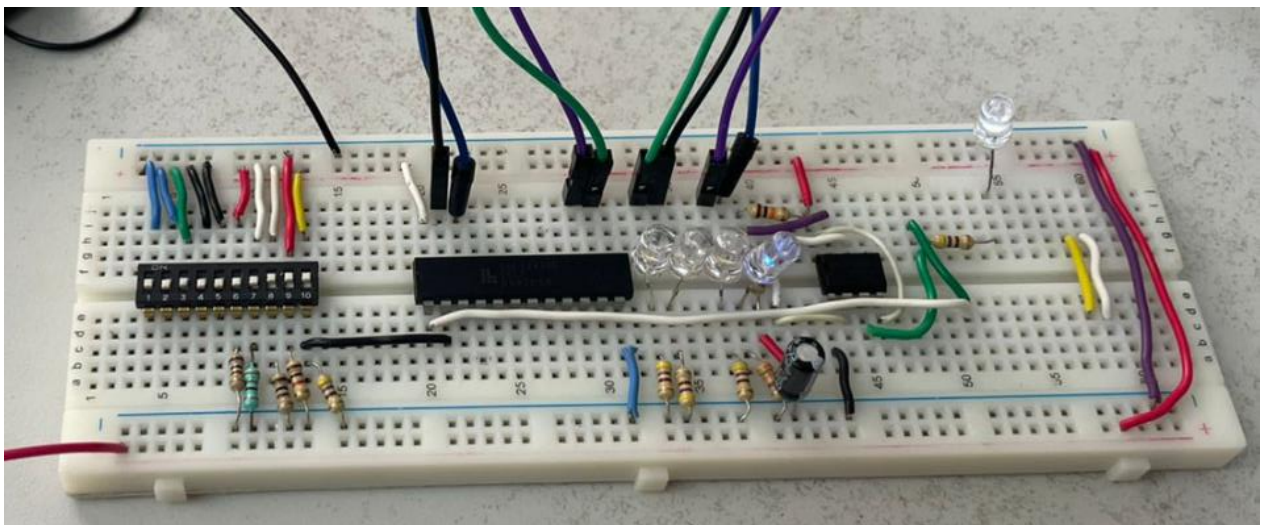
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity contador is
    port (clk: in std_logic;
          UD: in std_logic;
          Q: inout std_logic_vector(3 downto 0));
end contador;

architecture a_contador of contador is
begin
    process (UD, clk)
    begin
        if (clk'event and clk = '1') then
            if (UD = '0') then
                Q <= Q + 1;
            else
                Q <= Q - 1;
            end if;
        end if;
    end process;
end a_contador;
```

C22V10

clk =	1	24	* not used
ud =	2	23	= q(2)
not used *	3	22	= q(0)
not used *	4	21	* not used
not used *	5	20	* not used
not used *	6	19	* not used
not used *	7	18	* not used
not used *	8	17	* not used
not used *	9	16	* not used
not used *	10	15	= q(1)
not used *	11	14	= q(3)
not used *	12	13	* not used

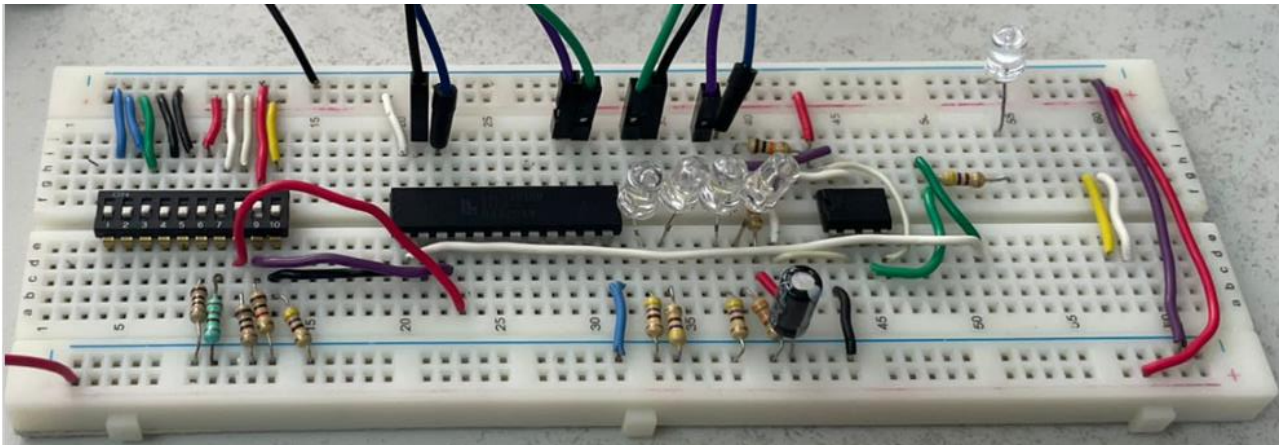


Contador de 4 bits con Reset y Carga en Paralelo

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
entity cont is
    port (P: in std_logic_vector(3 downto 0);
          clk, Load, Enp, Reset: in std_logic;
          Q: inout std_logic_vector(3 downto 0));
end cont;
architecture arq_cont of cont is
begin
    process (clk, Reset, Load, Enp)
    begin
        if (Reset = '1') then
            Q <= "0000"; -- Operación Asíncrona
        elsif (clk'event and clk = '1') then
            if (Load = '0' and Enp = '-') then
                Q <= P;
            elsif (Load = '1' and Enp = '0') then
                Q <= Q;
            elsif (Load = '1' and Enp = '1') then
                Q <= Q + 1;
            end if;
        end if;
    end process;
end arq_cont;
```


C22V10

clk = 1	24 * not used
reset = 2	23 = q(2)
load = 3	22 = q(0)
enp = 4	21 * not used
not used * 5	20 * not used
not used * 6	19 * not used
not used * 7	18 * not used
not used * 8	17 * not used
not used * 9	16 * not used
not used * 10	15 = q(1)
not used * 11	14 = q(3)
not used * 12	13 * not used



OBSERVACIONES Y CONCLUSIONES

Montealegre Rosales David Uriel

La implementación de contadores secuenciales ascendentes y descendentes, junto con funciones de reset y carga paralela, destacó la importancia de manejar el flujo de datos en sistemas sincronizados. El diseño de estos contadores me permitió explorar cómo los flancos del reloj, junto con las señales de control, afectan la lógica combinacional y secuencial. Esto me llevó a entender que las operaciones asíncronas, como el Reset, requieren un manejo preciso para evitar comportamientos indeterminados en sistemas digitales complejos. Además, fue evidente que el rango de los contadores está directamente relacionado con el ancho de los registros utilizados. Este detalle técnico es crucial para evitar saturaciones o errores en aplicaciones prácticas como sistemas industriales o temporizadores avanzados.

Vázquez Blancas Cesar Said

Durante esta práctica, comprendí la relevancia de trabajar con estructuras jerárquicas en diseño digital. El uso de VHDL para implementar un contador modular mostró cómo dividir un problema en subsistemas manejables mejora la claridad y la escalabilidad del diseño. Sin embargo, observé que la síntesis de estructuras en herramientas específicas puede no ser óptima, por lo que es necesario revisar las rutas críticas para garantizar un diseño eficiente en hardware. Adicionalmente, trabajar con señales como UD para cambiar entre modos ascendentes y descendentes demostró la importancia de validar estos comportamientos bajo condiciones límite, como altas frecuencias de reloj o señales con ruido, lo cual asegura un diseño robusto y confiable al implementarse en FPGAs.

Salauz Hernández Nerick Francisco

La práctica me permitió observar el impacto de los sistemas asíncronos y síncronos en el diseño digital. Particularmente, trabajar con señales como Reset y Load demostró la necesidad de controlar adecuadamente las condiciones iniciales y los cambios de estado en sistemas críticos. Durante la implementación, entendí que estas señales no solo definen el comportamiento lógico del contador, sino que también influyen en su capacidad para responder a escenarios de tiempo real, como los que se presentan en sistemas embebidos. Por otro lado, me di cuenta de que encapsular las señales de control en módulos independientes aumenta significativamente la modularidad y la reutilización del diseño en proyectos más grandes, promoviendo soluciones escalables y eficientes.

5. BIBLIOGRAFIA

- **Pedroni, V. (2004).** *Circuit Design with VHDL*. MIT Press.
- **Roth, C. H., & Kinney, L. L. (2013).** *Digital Systems Design Using VHDL* (3rd ed.). Cengage Learning.
- **IEEE Std 1076-2008.** *IEEE Standard VHDL Language Reference Manual*. IEEE.
- **Tocci, R. J., Widmer, N. S., & Moss, G. L. (2016).** *Digital Systems: Principles and Applications* (12th ed.). Pearson.